```java
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.io.File;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/**
 * Программный комплекс поддержки принятия решений по управлению резервными
 * источниками энергоснабжения объектов. Один файл, только стандартная библиотека Java.
 */
public class PowerBackupDss extends JFrame {

    // ---------------------------------------------------------------- модель данных

    record Load(String name, double kw, int prio, int from, int to) {}
    record Gen(String name, double kw, double fuelLph, double minRunH, double fuelPrice, double wearPerKwh) {}
    record Bat(String name, double capKwh, double maxChargeKw, double maxDischargeKw, double eff, double socStartPct) {}
    record Seg(int from, int to, String mode, double limitKw) {}

    static final String M_NORMAL = "Норма";
    static final String M_PARTIAL = "Частичное ограничение";
    static final String M_OUTAGE = "Отключение";

    record Settings(int horizonH, int stepMin, double gridKw, double tariff,
                    double[] penalty, double storageValue) {}

    static final double IDLE_FUEL_SHARE = 0.3;   // доля расхода топлива ДГУ на холостом ходу
    static final double BAT_RESERVE = 0.10;      // неснижаемый остаток АКБ
    static final String[] PRIO_NAMES = {"Критические", "Важные", "Второстепенные"};
    static final String[] BAT_MODES = {"Резерв", "Разряд", "Заряд"};

    static class Step {
        double t, gridLimit, grid, batKw, fuelL, demandKw, servedKw;
        double[] gen, demand = new double[3], served = new double[3];
        double costGrid, costFuel, costWear, costPenalty, socKwh, socPct;
        String mode, batMode;
        int gensOn;
    }

    // ---------------------------------------------------------------- моделирование

    static class Simulator {
        final List<Load> loads; final List<Gen> gens; final List<Bat> bats;
        final List<Seg> segs; final Settings s; final double dt;

        Simulator(List<Load> l, List<Gen> g, List<Bat> b, List<Seg> sg, Settings st) {
            loads = l; gens = g; bats = b; segs = sg; s = st; dt = st.stepMin() / 60.0;
        }

        Seg segAt(double t) {
            Seg r = null;
            for (Seg x : segs) if (t >= x.from() && t < x.to()) r = x;
            return r;
        }

        double gridLimit(double t) {
            Seg x = segAt(t);
            if (x == null || M_NORMAL.equals(x.mode())) return s.gridKw();
            if (M_OUTAGE.equals(x.mode())) return 0;
            return Math.min(s.gridKw(), x.limitKw());
        }

        String modeName(double t) { Seg x = segAt(t); return x == null ? M_NORMAL : x.mode(); }

        List<Step> run() {
            int n = (int) Math.round(s.horizonH() * 60.0 / s.stepMin());
            int ng = gens.size(), nb = bats.size();
            if (ng > 12) throw new IllegalArgumentException("Допускается не более 12 генераторов");
            double[] soc = new double[nb];
            for (int i = 0; i < nb; i++) soc[i] = bats.get(i).capKwh() * bats.get(i).socStartPct() / 100.0;
            int[] runLeft = new int[ng];
            boolean[] wasOn = new boolean[ng];
            List<Step> out = new ArrayList<>();
            for (int k = 0; k < n; k++) {
                double t = k * dt;
                Step best = null; int bestMask = 0, bestMode = 0; double bestCost = Double.MAX_VALUE;
                for (int mask = 0; mask < (1 << ng); mask++) {
                    boolean ok = true;
```

```
                    for (int i = 0; i < ng; i++) if (runLeft[i] > 0 && (mask >> i & 1) == 0) { ok = false; break; }
                    if (!ok) continue;
                    for (int mode = 0; mode < 3; mode++) {
                        Step st = eval(t, mask, mode, soc);
                        double c = totalCost(st, soc) + Integer.bitCount(mask) * 1e-6;
                        if (c < bestCost - 1e-9) { bestCost = c; best = st; bestMask = mask; bestMode = mode; }
                    }
                }
                commit(best, bestMask, bestMode, soc);
                for (int i = 0; i < ng; i++) {
                    boolean on = (bestMask >> i & 1) == 1;
                    if (on && !wasOn[i]) runLeft[i] = Math.max(0, (int) Math.ceil(gens.get(i).minRunH() / dt - 1e-9) - 1);
                    else if (on && runLeft[i] > 0) runLeft[i]--;
                    else if (!on) runLeft[i] = 0;
                    wasOn[i] = on;
                }
                best.socKwh = 0;
                for (double x : soc) best.socKwh += x;
                double cap = bats.stream().mapToDouble(Bat::capKwh).sum();
                best.socPct = cap > 0 ? best.socKwh / cap * 100 : 0;
                out.add(best);
            }
            return out;
        }

        double storedDelta;

        double totalCost(Step st, double[] soc) {
            return st.costGrid + st.costFuel + st.costWear + st.costPenalty - s.storageValue() * storedDelta;
        }

        /** Расчёт одного варианта управления: набор включённых ДГУ + режим АКБ. */
        Step eval(double t, int mask, int mode, double[] soc) {
            Step st = new Step();
            st.t = t; st.mode = modeName(t); st.batMode = BAT_MODES[mode];
            st.gridLimit = gridLimit(t);
            double genCap = 0;
            for (int i = 0; i < gens.size(); i++) if ((mask >> i & 1) == 1) genCap += gens.get(i).kw();
            double supplyCap = st.gridLimit + genCap;
            double disCap = 0, chRoom = 0;
            for (int i = 0; i < bats.size(); i++) {
                Bat b = bats.get(i);
                if (mode == 1) disCap += Math.min(b.maxDischargeKw(), Math.max(0, soc[i] - BAT_RESERVE * b.capKwh()) * b.eff() / dt);
                if (mode == 2) chRoom += Math.min(b.maxChargeKw(), Math.max(0, b.capKwh() - soc[i]) / (b.eff() * dt));
            }
            for (Load l : loads) if (t >= l.from() && t < l.to()) st.demand[l.prio() - 1] += l.kw();
            double avail = supplyCap + disCap, S = 0;
            for (int p = 0; p < 3; p++) {
                st.served[p] = Math.min(st.demand[p], avail);
                avail -= st.served[p]; S += st.served[p];
                st.demandKw += st.demand[p];
            }
            st.servedKw = S;
            double nonBat = Math.min(S, supplyCap), batDeliv = S - nonBat;
            double charge = mode == 2 ? Math.min(supplyCap - nonBat, chRoom) : 0;
            double P = nonBat + charge;
            st.grid = Math.min(P, st.gridLimit);
            double rest = P - st.grid;
            st.gen = new double[gens.size()];
            // загрузка ДГУ в порядке удельного расхода топлива
            Integer[] ord = new Integer[gens.size()];
            for (int i = 0; i < ord.length; i++) ord[i] = i;
            java.util.Arrays.sort(ord, (a, b) -> Double.compare(gens.get(a).fuelLph() / gens.get(a).kw(),
                                                                gens.get(b).fuelLph() / gens.get(b).kw()));
            for (int i : ord) {
                if ((mask >> i & 1) == 0) continue;
                Gen g = gens.get(i);
                st.gen[i] = Math.min(g.kw(), rest); rest -= st.gen[i];
                double frac = st.gen[i] / g.kw();
                double fuel = g.fuelLph() * (IDLE_FUEL_SHARE + (1 - IDLE_FUEL_SHARE) * frac) * dt;
                st.fuelL += fuel; st.costFuel += fuel * g.fuelPrice();
                st.costWear += g.wearPerKwh() * st.gen[i] * dt;
                st.gensOn++;
            }
            st.costGrid = st.grid * dt * s.tariff();
            for (int p = 0; p < 3; p++) st.costPenalty += (st.demand[p] - st.served[p]) * dt * s.penalty()[p];
            st.batKw = batDeliv - charge;
            double eff = bats.isEmpty() ? 1 : bats.get(0).eff();
            storedDelta = charge * dt * eff - batDeliv * dt / eff;
            st.mode = modeName(t);
```

```
            return st;
        }

        void commit(Step st, int mask, int mode, double[] soc) {
            double dis = Math.max(0, st.batKw), ch = Math.max(0, -st.batKw);
            double[] w = new double[bats.size()]; double sum = 0;
            for (int i = 0; i < bats.size(); i++) {
                Bat b = bats.get(i);
                w[i] = dis > 0 ? Math.min(b.maxDischargeKw(), Math.max(0, soc[i] - BAT_RESERVE * b.capKwh()) * b.eff() / dt)
                               : Math.min(b.maxChargeKw(), Math.max(0, b.capKwh() - soc[i]) / (b.eff() * dt));
                sum += w[i];
            }
            for (int i = 0; i < bats.size() && sum > 0; i++) {
                Bat b = bats.get(i);
                if (dis > 0) soc[i] -= dis * w[i] / sum * dt / b.eff();
                else if (ch > 0) soc[i] += ch * w[i] / sum * dt * b.eff();
            }
        }
    }

    // ------------------------------------------------------------------ интерфейс

    final DefaultTableModel loadsM = model(new String[]{"Нагрузка", "Мощность, кВт", "Приоритет (1-3)", "С часа", "До часа"},
            new Class<?>[]{String.class, Double.class, Integer.class, Integer.class, Integer.class});
    final DefaultTableModel gensM = model(new String[]{"Генератор", "Мощность, кВт", "Расход при полной нагрузке, л/ч",
            "Мин. время работы, ч", "Цена топлива, руб/л", "Износ, руб/кВт·ч"},
            new Class<?>[]{String.class, Double.class, Double.class, Double.class, Double.class, Double.class});
    final DefaultTableModel batsM = model(new String[]{"АКБ", "Ёмкость, кВт·ч", "Макс. заряд, кВт", "Макс. разряд, кВт",
            "КПД (0-1)", "Начальный заряд, %"},
            new Class<?>[]{String.class, Double.class, Double.class, Double.class, Double.class, Double.class});
    final DefaultTableModel segsM = model(new String[]{"С часа", "До часа", "Режим сети", "Предел мощности, кВт"},
            new Class<?>[]{Integer.class, Integer.class, String.class, Double.class});
    final DefaultTableModel stepsM = model(new String[]{"Время", "Сеть", "Предел сети, кВт", "Потребность, кВт", "Обеспечено, кВт",
            "Из сети, кВт", "ДГУ, кВт", "АКБ, кВт (+разряд)", "ДГУ вкл.", "Режим АКБ", "Заряд АКБ, %", "Топливо, л"},
            new Class<?>[]{String.class, String.class, Double.class, Double.class, Double.class, Double.class, Double.class,
                    Double.class, Integer.class, String.class, Double.class, Double.class});

    final JSpinner horizon = new JSpinner(new SpinnerNumberModel(24, 1, 168, 1));
    final JComboBox<Integer> stepMin = new JComboBox<>(new Integer[]{15, 30, 60});
    final JSpinner gridKw = new JSpinner(new SpinnerNumberModel(60.0, 0.0, 100000.0, 5.0));
    final JSpinner tariff = new JSpinner(new SpinnerNumberModel(6.0, 0.0, 1000.0, 0.5));
    final JSpinner[] penalty = {
            new JSpinner(new SpinnerNumberModel(200.0, 0.0, 100000.0, 10.0)),
            new JSpinner(new SpinnerNumberModel(60.0, 0.0, 100000.0, 5.0)),
            new JSpinner(new SpinnerNumberModel(15.0, 0.0, 100000.0, 1.0))};
    final JSpinner storageValue = new JSpinner(new SpinnerNumberModel(20.0, 0.0, 100000.0, 1.0));

    final JTextArea report = new JTextArea();
    final ChartPanel powerChart = new ChartPanel(false);
    final ChartPanel socChart = new ChartPanel(true);
    final JTabbedPane tabs = new JTabbedPane();

    static DefaultTableModel model(String[] cols, Class<?>[] types) {
        return new DefaultTableModel(cols, 0) {
            @Override public Class<?> getColumnClass(int c) { return types[c]; }
            @Override public boolean isCellEditable(int r, int c) { return this != null && !cols[0].equals("Время"); }
        };
    }

    PowerBackupDss() {
        super("Поддержка принятия решений: резервное энергоснабжение объектов");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(1200, 780);
        setLocationRelativeTo(null);

        tabs.addTab("1. Нагрузки", tablePanel(loadsM, new Object[]{"Новая нагрузка", 5.0, 2, 0, 24},
                "Состав и мощность нагрузок. Приоритет: 1 — критические, 2 — важные, 3 — второстепенные. "
                        + "Нагрузка действует в часы [С; До)."));
        JPanel src = new JPanel(new GridLayout(2, 1, 0, 8));
        src.add(tablePanel(gensM, new Object[]{"ДГУ", 20.0, 6.0, 1.0, 60.0, 2.0}, "Генераторы"));
        src.add(tablePanel(batsM, new Object[]{"АКБ", 50.0, 20.0, 20.0, 0.92, 80.0}, "Аккумуляторные батареи"));
        tabs.addTab("2. Резервные источники", src);
        tabs.addTab("3. Сеть и сценарий", scenarioPanel());
        tabs.addTab("4. Результаты", resultPanel());

```

```
        JButton run = new JButton("▶ Рассчитать");
        run.addActionListener(e -> runSimulation());
        JButton example = new JButton("Загрузить пример");
        example.addActionListener(e -> loadExample());
        JButton export = new JButton("Сохранить отчёт…");
        export.addActionListener(e -> exportReport());
        JToolBar bar = new JToolBar();
        bar.setFloatable(false);
        bar.add(run); bar.addSeparator(); bar.add(example); bar.add(export);

        setLayout(new BorderLayout());
        add(bar, BorderLayout.NORTH);
        add(tabs, BorderLayout.CENTER);
        loadExample();
    }

    JPanel tablePanel(DefaultTableModel m, Object[] newRow, String hint) {
        JTable table = new JTable(m);
        table.setRowHeight(24);
        table.setFillsViewportHeight(true);
        JButton add = new JButton("Добавить");
        add.addActionListener(e -> m.addRow(newRow.clone()));
        JButton del = new JButton("Удалить");
        del.addActionListener(e -> {
            if (table.isEditing()) table.getCellEditor().stopCellEditing();
            int[] rows = table.getSelectedRows();
            for (int i = rows.length - 1; i >= 0; i--) m.removeRow(rows[i]);
        });
        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.LEFT));
        buttons.add(new JLabel(hint)); buttons.add(add); buttons.add(del);
        JPanel p = new JPanel(new BorderLayout());
        p.setBorder(new EmptyBorder(6, 6, 6, 6));
        p.add(buttons, BorderLayout.NORTH);
        p.add(new JScrollPane(table), BorderLayout.CENTER);
        return p;
    }

    JPanel scenarioPanel() {
        JPanel params = new JPanel(new GridBagLayout());
        params.setBorder(BorderFactory.createTitledBorder("Параметры моделирования"));
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(3, 6, 3, 6); c.anchor = GridBagConstraints.WEST;
        Object[][] rows = {
                {"Горизонт моделирования, ч", horizon}, {"Временной шаг, мин", stepMin},
                {"Мощность сети в норме, кВт", gridKw}, {"Тариф сети, руб/кВт·ч", tariff},
                {"Штраф за недообеспечение: критические, руб/кВт·ч", penalty[0]},
                {"Штраф за недообеспечение: важные, руб/кВт·ч", penalty[1]},
                {"Штраф за недообеспечение: второстепенные, руб/кВт·ч", penalty[2]},
                {"Ценность запаса энергии в АКБ, руб/кВт·ч", storageValue}};
        for (int i = 0; i < rows.length; i++) {
            c.gridy = i; c.gridx = 0; params.add(new JLabel((String) rows[i][0]), c);
            c.gridx = 1; JComponent comp = (JComponent) rows[i][1];
            comp.setPreferredSize(new Dimension(110, 24));
            params.add(comp, c);
        }
        JPanel segs = tablePanel(segsM, new Object[]{0, 1, M_OUTAGE, 0.0},
                "Сценарий внешней сети (вне интервалов — нормальное электроснабжение)");
        JTable t = (JTable) ((JScrollPane) ((BorderLayout) segs.getLayout()).getLayoutComponent(BorderLayout.CENT
ER)).getViewport().getView();
        t.getColumnModel().getColumn(2).setCellEditor(new DefaultCellEditor(
                new JComboBox<>(new String[]{M_NORMAL, M_PARTIAL, M_OUTAGE})));
        JPanel p = new JPanel(new BorderLayout(0, 8));
        p.setBorder(new EmptyBorder(6, 6, 6, 6));
        JPanel west = new JPanel(new BorderLayout());
        west.add(params, BorderLayout.NORTH);
        p.add(west, BorderLayout.WEST);
        p.add(segs, BorderLayout.CENTER);
        return p;
    }

    JComponent resultPanel() {
        report.setEditable(false);
        report.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));
        JPanel charts = new JPanel(new GridLayout(2, 1, 0, 6));
        charts.add(powerChart); charts.add(socChart);
        JTable steps = new JTable(stepsM);
        steps.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
        steps.setRowHeight(22);
        for (int i = 0; i < steps.getColumnCount(); i++) steps.getColumnModel().getColumn(i).setPreferredWidth(i
== 1 ? 150 : 115);
        JTabbedPane inner = new JTabbedPane(JTabbedPane.BOTTOM);
        inner.addTab("Отчёт", new JScrollPane(report));
        inner.addTab("Графики", charts);
        inner.addTab("Решения по шагам", new JScrollPane(steps));
```

```
        return inner;
    }

    void loadExample() {
        loadsM.setRowCount(0); gensM.setRowCount(0); batsM.setRowCount(0); segsM.setRowCount(0);
        Object[][] l = {{"Медицинский блок", 15.0, 1, 0, 24}, {"Серверная", 10.0, 1, 0, 24},
                {"Освещение", 8.0, 2, 0, 24}, {"Вентиляция", 12.0, 2, 6, 22},
                {"Бытовые потребители", 25.0, 3, 8, 20}, {"Лаборатория", 18.0, 3, 9, 18}};
        for (Object[] r : l) loadsM.addRow(r);
        gensM.addRow(new Object[]{"ДГУ-1", 30.0, 9.0, 2.0, 60.0, 2.0});
        gensM.addRow(new Object[]{"ДГУ-2", 20.0, 6.0, 1.0, 60.0, 2.0});
        batsM.addRow(new Object[]{"АКБ-1", 80.0, 25.0, 30.0, 0.92, 90.0});
        segsM.addRow(new Object[]{6, 8, M_PARTIAL, 25.0});
        segsM.addRow(new Object[]{10, 16, M_OUTAGE, 0.0});
        segsM.addRow(new Object[]{19, 22, M_PARTIAL, 20.0});
        horizon.setValue(24); stepMin.setSelectedItem(60);
    }

    // ---------------------------------------------------------------- чтение ввода

    static void stop(JTable t) { if (t != null && t.isEditing()) t.getCellEditor().stopCellEditing(); }
    static double num(DefaultTableModel m, int r, int c) { return ((Number) m.getValueAt(r, c)).doubleValue(); }

    void stopAllEditing() {
        for (Component c : allTables(this)) stop((JTable) c);
    }

    static List<Component> allTables(Container root) {
        List<Component> res = new ArrayList<>();
        for (Component c : root.getComponents()) {
            if (c instanceof JTable) res.add(c);
            else if (c instanceof Container ct) res.addAll(allTables(ct));
        }
        return res;
    }

    Simulator readInput() {
        stopAllEditing();
        List<Load> loads = new ArrayList<>(); List<Gen> gens = new ArrayList<>();
        List<Bat> bats = new ArrayList<>(); List<Seg> segs = new ArrayList<>();
        int h = (Integer) horizon.getValue();
        for (int r = 0; r < loadsM.getRowCount(); r++) {
            Load l = new Load(String.valueOf(loadsM.getValueAt(r, 0)), num(loadsM, r, 1), (int) num(loadsM, r, 2)
,
                    (int) num(loadsM, r, 3), (int) num(loadsM, r, 4));
            if (l.kw() < 0 || l.prio() < 1 || l.prio() > 3 || l.from() < 0 || l.to() <= l.from())
                throw new IllegalArgumentException("Некорректная нагрузка «" + l.name() + "» (строка " + (r + 1)
+ ")");
            loads.add(l);
        }
        for (int r = 0; r < gensM.getRowCount(); r++) {
            Gen g = new Gen(String.valueOf(gensM.getValueAt(r, 0)), num(gensM, r, 1), num(gensM, r, 2),
                    num(gensM, r, 3), num(gensM, r, 4), num(gensM, r, 5));
            if (g.kw() <= 0 || g.fuelLph() < 0 || g.minRunH() < 0)
                throw new IllegalArgumentException("Некорректный генератор «" + g.name() + "» (строка " + (r + 1)
 + ")");
            gens.add(g);
        }
        for (int r = 0; r < batsM.getRowCount(); r++) {
            Bat b = new Bat(String.valueOf(batsM.getValueAt(r, 0)), num(batsM, r, 1), num(batsM, r, 2),
                    num(batsM, r, 3), num(batsM, r, 4), num(batsM, r, 5));
            if (b.capKwh() <= 0 || b.eff() <= 0 || b.eff() > 1 || b.socStartPct() < 0 || b.socStartPct() > 100)
                throw new IllegalArgumentException("Некорректная АКБ «" + b.name() + "» (строка " + (r + 1) + ")"
);
            bats.add(b);
        }
        for (int r = 0; r < segsM.getRowCount(); r++) {
            Seg s = new Seg((int) num(segsM, r, 0), (int) num(segsM, r, 1), String.valueOf(segsM.getValueAt(r, 2)
), num(segsM, r, 3));
            if (s.to() <= s.from()) throw new IllegalArgumentException("Некорректный интервал сети (строка " + (r
 + 1) + ")");
            segs.add(s);
        }
        double[] pen = {(Double) penalty[0].getValue(), (Double) penalty[1].getValue(), (Double) penalty[2].getVa
lue()};
        Settings st = new Settings(h, (Integer) stepMin.getSelectedItem(), (Double) gridKw.getValue(),
                (Double) tariff.getValue(), pen, (Double) storageValue.getValue());
        return new Simulator(loads, gens, bats, segs, st);
    }

    // ---------------------------------------------------------------- расчёт и вывод

    List<Step> lastSteps; Simulator lastSim;

```

```
    void runSimulation() {
        try {
            Simulator sim = readInput();
            List<Step> steps = sim.run();
            lastSim = sim; lastSteps = steps;
            fillSteps(sim, steps);
            report.setText(buildReport(sim, steps));
            report.setCaretPosition(0);
            powerChart.setData(sim, steps);
            socChart.setData(sim, steps);
            tabs.setSelectedIndex(3);
        } catch (IllegalArgumentException | ClassCastException | NullPointerException ex) {
            JOptionPane.showMessageDialog(this, "Ошибка во входных данных: " + ex.getMessage(),
                    "Расчёт невозможен", JOptionPane.ERROR_MESSAGE);
        }
    }

    static String clock(double t) { int m = (int) Math.round(t * 60); return String.format("%02d:%02d", m / 60, m % 60); }

    void fillSteps(Simulator sim, List<Step> steps) {
        stepsM.setRowCount(0);
        for (Step s : steps) {
            double g = 0; for (double x : s.gen) g += x;
            stepsM.addRow(new Object[]{clock(s.t), s.mode, s.gridLimit, s.demandKw, s.servedKw, s.grid, g, s.batKw,
                    s.gensOn, s.batMode, s.socPct, s.fuelL});
        }
    }

    String buildReport(Simulator sim, List<Step> steps) {
        double dt = sim.dt;
        double[] dem = new double[3], srv = new double[3];
        double fuel = 0, cGrid = 0, cFuel = 0, cWear = 0, cPen = 0, eGrid = 0, eGen = 0, eBatOut = 0, eBatIn = 0;
        double[] genE = new double[sim.gens.size()];
        int outageSteps = 0;
        for (Step s : steps) {
            for (int p = 0; p < 3; p++) { dem[p] += s.demand[p] * dt; srv[p] += s.served[p] * dt; }
            fuel += s.fuelL; cGrid += s.costGrid; cFuel += s.costFuel; cWear += s.costWear; cPen += s.costPenalty;
            eGrid += s.grid * dt;
            for (int i = 0; i < s.gen.length; i++) { genE[i] += s.gen[i] * dt; eGen += s.gen[i] * dt; }
            if (s.batKw > 0) eBatOut += s.batKw * dt; else eBatIn -= s.batKw * dt;
            if (s.gridLimit < sim.s.gridKw() - 1e-9) outageSteps++;
        }
        double dTot = dem[0] + dem[1] + dem[2], sTot = srv[0] + srv[1] + srv[2];
        double cov = dTot > 0 ? sTot / dTot * 100 : 100;
        double covCrit = dem[0] > 0 ? srv[0] / dem[0] * 100 : 100;
        String stab = covCrit >= 99.999 && cov >= 90 ? "ВЫСОКАЯ — критические нагрузки обеспечены полностью"
                : covCrit >= 90 ? "СРЕДНЯЯ — критические нагрузки обеспечены частично"
                : "НИЗКАЯ — критические нагрузки не обеспечены";
        StringBuilder b = new StringBuilder();
        Locale L = Locale.US;
        b.append(String.format(L, "ИТОГОВЫЙ ОТЧЁТ МОДЕЛИРОВАНИЯ%nГоризонт: %d ч, шаг: %d мин, шагов: %d (с ограничением сети: %d)%n%n",
                sim.s.horizonH(), sim.s.stepMin(), steps.size(), outageSteps));
        b.append("1. Обеспечение электроэнергией, кВт·ч\n");
        b.append(String.format(L, "   Потребность:       %10.1f%n   Обеспечено:        %10.1f%n   Недообеспечено: %10.1f%n%n",
                dTot, sTot, dTot - sTot));
        b.append("2. Уровень покрытия нагрузок по приоритетам\n");
        for (int p = 0; p < 3; p++)
            b.append(String.format(L, "   %-16s потребность %8.1f, обеспечено %8.1f, покрытие %6.1f %%%n",
                    PRIO_NAMES[p], dem[p], srv[p], dem[p] > 0 ? srv[p] / dem[p] * 100 : 100.0));
        b.append(String.format(L, "   Суммарное покрытие: %.1f %%%n%n", cov));
        b.append("3. Источники энергии, кВт·ч\n");
        b.append(String.format(L, "   Внешняя сеть: %.1f%n", eGrid));
        for (int i = 0; i < genE.length; i++) b.append(String.format(L, "   %s: %.1f%n", sim.gens.get(i).name(), genE[i]));
        b.append(String.format(L, "   АКБ: отдано %.1f, получено на заряд %.1f%n   Расход топлива ДГУ: %.1f л%n%n", eBatOut, eBatIn, fuel));
        b.append("4. Стоимость, руб\n");
        b.append(String.format(L, "   Топливо:               %10.1f%n   Износ ДГУ:             %10.1f%n   Электроэнергия сети:   %10.1f%n"
                        + "   Штрафы недообеспечения:%10.1f%n   ИТОГО:                 %10.1f%n%n",
                cFuel, cWear, cGrid, cPen, cFuel + cWear + cGrid + cPen));
        double socEnd = steps.isEmpty() ? 0 : steps.get(steps.size() - 1).socPct;
        b.append(String.format(L, "5. Заряд АКБ в конце периода: %.1f %%%n%n", socEnd));
        b.append("6. Оценка устойчивости электроснабжения: ").append(stab).append('\n');
        return b.toString();
    }

    void exportReport() {
        if (lastSteps == null) { runSimulation(); if (lastSteps == null) return; }
```

```
        JFileChooser fc = new JFileChooser();
        fc.setSelectedFile(new File("report.txt"));
        if (fc.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) return;
        try {
            Files.writeString(fc.getSelectedFile().toPath(), report.getText(), StandardCharsets.UTF_8);
        } catch (java.io.IOException ex) {
            JOptionPane.showMessageDialog(this, "Не удалось сохранить: " + ex.getMessage(), "Ошибка", JOptionPane.ERROR_MESSAGE);
        }
    }

    // ---------------------------------------------------------------- графики

    static class ChartPanel extends JPanel {
        final boolean soc; Simulator sim; List<Step> steps;
        static final Color GRID = new Color(76, 153, 94), GEN = new Color(230, 140, 40), BAT = new Color(60, 120, 200),
                LOST = new Color(205, 60, 60), OUT = new Color(235, 235, 235);

        ChartPanel(boolean soc) { this.soc = soc; setBackground(Color.WHITE); setPreferredSize(new Dimension(600, 250)); }

        void setData(Simulator s, List<Step> st) { sim = s; steps = st; repaint(); }

        @Override protected void paintComponent(Graphics g0) {
            super.paintComponent(g0);
            Graphics2D g = (Graphics2D) g0;
            g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
            int w = getWidth(), h = getHeight(), l = 60, r = 16, t = 28, bt = 30;
            g.setColor(Color.DARK_GRAY);
            g.drawString(soc ? "Заряд аккумуляторных батарей, %" : "Баланс мощности по шагам, кВт", l, 16);
            if (steps == null || steps.isEmpty()) { g.drawString("Нет данных — нажмите «Рассчитать»", l, h / 2); return; }
            int n = steps.size();
            double max = soc ? 100 : 1;
            if (!soc) for (Step s : steps) max = Math.max(max, Math.max(s.demandKw, s.grid + sum(s.gen) + Math.max(0, s.batKw)));
            if (!soc) max *= 1.1;
            double pw = (w - l - r) / (double) n, ph = h - t - bt;
            for (int i = 0; i < n; i++) {                    // фон: ограничения сети
                Step s = steps.get(i);
                if (s.gridLimit < sim.s.gridKw() - 1e-9) { g.setColor(OUT); g.fillRect((int) (l + i * pw), t, (int) Math.ceil(pw), (int) ph); }
            }
            g.setColor(Color.LIGHT_GRAY);
            for (int k = 0; k <= 4; k++) {
                int y = (int) (t + ph - ph * k / 4);
                g.drawLine(l, y, w - r, y);
                g.setColor(Color.DARK_GRAY); g.drawString(String.format(Locale.US, "%.0f", max * k / 4), 8, y + 4); g.setColor(Color.LIGHT_GRAY);
            }
            if (soc) {
                g.setColor(BAT); g.setStroke(new BasicStroke(2f));
                int px = 0, py = 0;
                for (int i = 0; i < n; i++) {
                    int x = (int) (l + (i + 0.5) * pw), y = (int) (t + ph - ph * steps.get(i).socPct / max);
                    if (i > 0) g.drawLine(px, py, x, y);
                    px = x; py = y;
                }
            } else {
                for (int i = 0; i < n; i++) {
                    Step s = steps.get(i);
                    double base = 0; int x = (int) (l + i * pw + pw * 0.1), bw = Math.max(1, (int) (pw * 0.8));
                    double[] vals = {s.grid, sum(s.gen), Math.max(0, s.batKw), s.demandKw - s.servedKw};
                    Color[] cols = {GRID, GEN, BAT, LOST};
                    for (int k = 0; k < 4; k++) {
                        int hh = (int) Math.round(ph * vals[k] / max);
                        g.setColor(cols[k]); g.fillRect(x, (int) (t + ph - ph * base / max - hh), bw, hh);
                        base += vals[k];
                    }
                }
                g.setColor(Color.BLACK); g.setStroke(new BasicStroke(1.6f));
                int px = 0, py = 0;
                for (int i = 0; i < n; i++) {
                    int x = (int) (l + (i + 0.5) * pw), y = (int) (t + ph - ph * steps.get(i).demandKw / max);
                    if (i > 0) g.drawLine(px, py, x, y);
                    px = x; py = y;
                }
                String[] names = {"Сеть", "ДГУ", "АКБ", "Недообеспечено", "Потребность"};
                Color[] cols = {GRID, GEN, BAT, LOST, Color.BLACK};
                int lx = l + 200;
                for (int k = 0; k < 5; k++) {
                    g.setColor(cols[k]); g.fillRect(lx, 6, 10, 10);
                    g.setColor(Color.DARK_GRAY); g.drawString(names[k], lx + 14, 15); lx += 28 + g.getFontMetrics
```

```
    ().stringWidth(names[k]);
551                 }
552             }
553             g.setStroke(new BasicStroke(1f)); g.setColor(Color.DARK_GRAY);
554             g.drawLine(l, (int) (t + ph), w - r, (int) (t + ph));
555             int every = Math.max(1, n / 12);
556             for (int i = 0; i < n; i += every) g.drawString(clock(steps.get(i).t), (int) (l + i * pw), h - 10);
557             g.drawString("(серый фон — ограничение или отключение сети)", w - 330, h - 10);
558         }
559 
560         static double sum(double[] a) { double s = 0; for (double x : a) s += x; return s; }
561     }
562 
563     public static void main(String[] args) {
564         Locale.setDefault(Locale.US);
565         SwingUtilities.invokeLater(() -> {
566             try { UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()); } catch (Exception ignored
    ) { }
567             new PowerBackupDss().setVisible(true);
568         });
569     }
570 }
571 
```